

The Vivarium Semantic Modeling Layer

A backend-neutral layer for biological meaning over the Process-Bigraph runtime · grounded against the working codebase

ABSTRACT

Vivarium already guarantees that a model is well-typed, executable, and behaves as expected. It does not yet capture, in a portable and machine-checkable form, what a model *asserts about biology*. This document specifies a thin layer — a biological-claim vocabulary and annotation over primitives Vivarium already ships — that makes the biological claim a first-class, versioned, backend-neutral artifact: one that constrains its realizations, records its own approximations honestly, and links from claim to evidence. It is grounded against the current codebase; roughly 70% of the proposal is naming and single-sourcing what already exists. The claim-to-runtime *compiler* is deferred as a research direction. Chemotactic recruitment serves as the worked example throughout.

STATUS Proposed · grounded against shipping code	RUNTIME SUBSTRATE Process-Bigraph (Vivarium 2.0), unchanged
PRIMARY HOME viva_superpowers + biological-claim/0.1	REFERENCE SLICE Chemotactic recruitment (already prototyped)

HOW TO READ THIS DOCUMENT

1. **Part I — Background.** What Vivarium is today, the gap this addresses, and the goals. Start here if Vivarium is new to you.

2. **Part II — The plan.** What the layer is, mapped to code that already exists, and the roadmap.

3. **Part III — Worked case study.** Chemotactic recruitment end to end with the real files, then the same slice once modeling is semantic-driven.

4. **Part IV — Reference & North Star.** Closure predicates, architecture, the long-term vision, non-goals, and corrections.

Note on this revision (0.1 → 0.2). The prior draft read as a greenfield architecture. This revision maps every pillar to the shipping code that implements it, corrects the packages and mechanisms it misnamed

(Appendix, §14), and demotes the compiler to research. The Claim / Mechanism / Realization model is not proposed here — it is already prototyped on disk for the reference slice.

PART I

Background — Vivarium as it is today

The framework this layer sits on, the gap it fills, and what success looks like.

1 ORIENTATION

What Vivarium is

A framework for building biological models by composing independent parts over shared, typed state.

Vivarium is a compositional framework for multiscale biological simulation. Rather than one monolithic program, a model is assembled from independent *processes* — an ODE solver here, an agent-based spatial model there, a metabolic flux solver alongside — each wired to the others through a shared, schema-typed state. Its stated aim is to integrate models built in different formalisms (ODEs, flux-balance analysis, agent-based, spatial solvers, machine-learning components) into a single coherent simulation. It is a composition protocol, not a domain-specific simulator.

The current generation, **Vivarium 2.0**, rests on two libraries, with a formal account in Agmon & Spangler, *Process Bigraphs and the Architecture of Compositional Systems Biology* (arXiv:2512.23754):

RUNTIME

process-bigraph

The execution core. Declares process interfaces, wires processes through typed shared state, and orchestrates execution across heterogeneous timescales — including dynamic structure such as division and graph rewrites.

TYPE SYSTEM

bigraph-schema

A serializable schema language for hierarchical, validated state. Supplies types, units (`pint` -backed), bounded ranges (`Range{ _min, _max }`), enums, validation, and serialization.

The three runtime objects

Process

A temporal unit of computation: it reads state and advances it over an interval, `update(state, interval)`. It never mutates state directly — it emits a typed delta the runtime merges.

Step

A stateless transformation triggered when its inputs are ready — like a reaction or a derived-quantity rule, `update(state)`.

Composite	A network of Processes and Steps wired through a shared state tree and a bridge. A Composite is itself a Process, so composites nest — this is how multiscale models are built.
------------------	---

Because processes emit typed deltas rather than writing state, numerical updates, structural rewrites, and scheduling coexist under one execution semantics. A composite is an ordinary document — a tree of typed stores and wired process nodes — that can be serialized, versioned, and replayed.

Around the runtime

Two further layers turn the runtime into a research environment, and they matter here because the semantic layer lives among them:

- **The vivarium-workbench** — a dashboard for authoring composites, running simulations, and inspecting results. It also serves the published study reports.
- **The study / investigation layer** (in `viva_superpowers`) — turns raw simulations into scientific arguments. A *Study* carries a hypothesis, acceptance criteria, backend-neutral behavior tests, runs, findings, and a gate verdict; an *Investigation* groups studies under one research question. `viva-cpm` — a Rust Cellular-Potts engine wrapped as processes — is one such workspace, and the home of our case study.

The intended division of labor

The design behind this layer keeps each existing part of the ecosystem in a distinct role and adds one new artifact on top, rather than overloading any backend to carry biological meaning:

LAYER	ROLE
<code>bigraph-schema</code>	Generic type, document, graph, validation, transformation, and serialization substrate.
<code>process-bigraph</code>	Executable intermediate representation, component linker, orchestrator, and runtime.
<code>vivarium-workbench</code>	Conceptual-modeling studio, research notebook, evidence viewer, and human–AI collaboration environment.
<code>investigation-contracts</code>	Provenance, epistemic lifecycle, and evidence graphs.
<code>viva-marketplace</code>	Capability and implementation registry (today a repository/artifact ledger).
Biological assertion layer (new)	Backend-independent source language for claims, contexts, potential mechanisms, interventions, observables, and applicability.

Why Vivarium is well suited

The runtime already provides structures a biological-meaning layer would otherwise have to invent. The layer *formalizes and generalizes what is already emerging* rather than entering an ecosystem concerned only with runtime wiring:

- **Schema is separated from instantiated state.** Process-Bigraph distinguishes a schema — variables, types, interfaces, permitted connections — from a state of actual values for a run. That maps onto the distinction a meaning layer needs: a *potential-model template* versus an *instantiated world*.
- **Place graph and link graph give containment and connectivity.** A recursive place graph (organism ▶ tissue ▶ compartment ▶ cells) plus a hypergraph of interactions that cross those boundaries — so the environment is first-class without becoming one monolithic "everything else" object.
- **Typed updates begin a contribution algebra.** Processes emit typed deltas the runtime merges — the right foundation for simultaneous biological influences: forces add, fluxes add, rates combine, event hazards compete, constraints are jointly imposed.
- **Composites support encapsulation and refinement.** An atomic process can be replaced by a detailed composite exposing the same interface — the execution-level basis for refinement, missing only the *scientific* replacement contract.
- **Sites and fills prefigure gradual closure.** Templates carry typed sites (holes); a document becomes executable once required sites are filled — a strong precedent for gradual contracts and claim-scoped closure.
- **An epistemic layer is already emerging.** Studies, investigations, `given-measure-expect` behavior tests, and `investigation-contracts` (actors, provenance, findings, evidence, decisions) mean the layer generalizes current practice rather than replacing it.

2 MOTIVATION

The gap: running is not the same as meaning

Vivarium already guarantees a great deal. The type system guarantees a model is well-formed and executable. The study layer checks it behaves as expected — that a run reproduces a target measurement within a declared band. But neither captures, in a portable and checkable form, what the model *asserts about biology*.

Today that biological meaning lives implicitly: in a composite's parameter names, in prose scattered across several files, and in the modeler's head. Three concrete symptoms:

SYMPTOM

The claim drifts

One biological claim is written four ways — a structured bundle, the study `claim:`, the study `hypothesis:`, the investigation `main_claim` — and they fall out of sync. Nothing is the single source.

SYMPTOM

The gap is hidden

A CPM `lambda` is a phenomenological stand-in for receptor kinetics — but there is no machine-checkable place to say so, so the approximation silently reads as the real mechanism.

SYMPTOM

Meaning is not portable

One cannot ask "what else realizes this claim?" or "does this PhysiCell model make the same assertion as that CPM one?" — the claim is not a first-class, backend-neutral object.

Where current Vivarium stops

These gaps are not failures of Vivarium — most are exactly what one expects of a composition runtime. But they mean that simply attaching biological names to ports will not solve the conceptual-modeling problem. Each current construct handles something well and leaves something for a meaning layer to carry:

CURRENT CONSTRUCT	WHAT IT HANDLES WELL	WHAT A CONCEPTUAL LANGUAGE STILL NEEDS
Type schema	Data type, shape, units, defaults, update behavior	Biological meaning, referent identity, measurement meaning, uncertainty, context
Process port	Read / write interface	Affordance, controls, assumptions, legal protocol, evidence
Site and face	Structural compatibility of a filler	Semantic compatibility and claim-relative adequacy
Groundness	Whether a document can run	Whether a scientific claim is sufficiently specified
Composite	Computational encapsulation	Claim-preserving abstraction and refinement
Study	Run, readout, expected behavior	Formal hypothesis, intervention semantics, alternatives, applicability
Registry	Repositories, processes, descriptions	Searchable capabilities, evidence, valid ranges, semantic mappings
Runtime result	State trajectory	Causal history, applicability excursions, claim-level evidence

Port compatibility is not biological compatibility

Interface matching asks whether data shapes, types, units, and faces agree — and `bigraph-schema` already performs unit-aware conversion. But two quantities can share a numerical type and units while meaning different biology:

```
# identical type + units, different biological referent
cell volume as geometric occupied volume
cell volume as biochemical dilution volume
cell volume as target volume in a constraint energy
cell volume as an image-segmentation estimate
```

A `vector[3]` "polarity" could equally be an apicobasal axis, a front–rear migration axis, a principal geometric axis, or a noisy imaging estimate — the type cannot tell them apart. Hence the governing principle:

Principle. A Process-Bigraph process should be *one possible realization* of a biological mechanism, not the definition of that mechanism. Otherwise the conceptual language inherits the current component decomposition and we are back to package-centered modeling.

Thesis. Add a thin layer *above* the runtime that makes the biological claim a first-class, versioned, backend-neutral artifact — one that constrains its realizations, records its own approximations honestly, and links down to the evidence. Keep Process-Bigraph exactly as it is: the executable substrate, never redefined as the biology-assertion language. In the source notes' analogy, *the biological assertion language is the source language; Process-Bigraph is something like LLVM, a linker, and an execution runtime* — preserving the ability to target Vivarium-native models, CompuCell3D, PhysiCell, or generated code without letting any one backend decide what counts as a biological concept.

3 WHAT SUCCESS LOOKS LIKE

Goals

GOAL

Minimal intrusion

Existing `Process`, `Composite`, and `Step` semantics stay valid. A component with no semantic metadata still runs; it is simply *unknown* to the meaning layer, never broken by it.

GOAL

Single source of truth

One canonical claim, from which study and investigation prose are derived. Drift becomes impossible by construction rather than caught by review.

GOAL

Backend independence

Claims and mechanisms are defined without reference to CPM, PhysiCell, or any engine — so one claim can be realized, and tested, across several backends.

GOAL

Honest approximation

Every realization records its fidelity and its semantic gap — what the backend cannot express — as structured fields, not buried prose.

GOAL

Traceability

From claim to mechanism to realization to executable artifact to run to observable to evidence, every link is queryable — reusing the linkage graph that already ships.

GOAL

Reuse over reinvention

Provenance, acceptance lifecycle, ranges, and verdicts already exist. Adopt `investigation-contracts` and `bigraph-schema` primitives instead of parallel ones.

PART II

The plan — a thin layer over what already ships

What the semantic layer is, where each piece already lives, and how it rolls out.

4 THE SHAPE OF THE LAYER

Three documents and a verdict

The layer adds three small, versioned, backend-neutral documents above each composite, plus one derived verdict. The chemotactic-recruitment investigation already carries a hand-authored 0.1 prototype of exactly this, so the work is to *formalize* it, not invent it.

Biological Claim	<i>What is asserted</i> — a subject–relation–object assertion, its context, the intervention that should abolish it, the observable that measures it, its alternatives and falsifiers. Backend-free.
Conceptual Mechanism	<i>What can happen</i> — the roles and capabilities that could realize the claim (a "process schema"), with the controls it exposes. Says nothing of pixels, lattices, or forces.
Realization	<i>How it is made executable</i> in one backend — role-to-component bindings, per-binding fidelity, inherited defaults, explicit exclusions, and the honest semantic gap.
Closure (derived)	A code-computed verdict — <code>well_formed</code> / <code>claim_complete</code> / <code>backend_realizable</code> / <code>execution_ground</code> — over signals that already exist. Never hand-set. Detailed in §9.

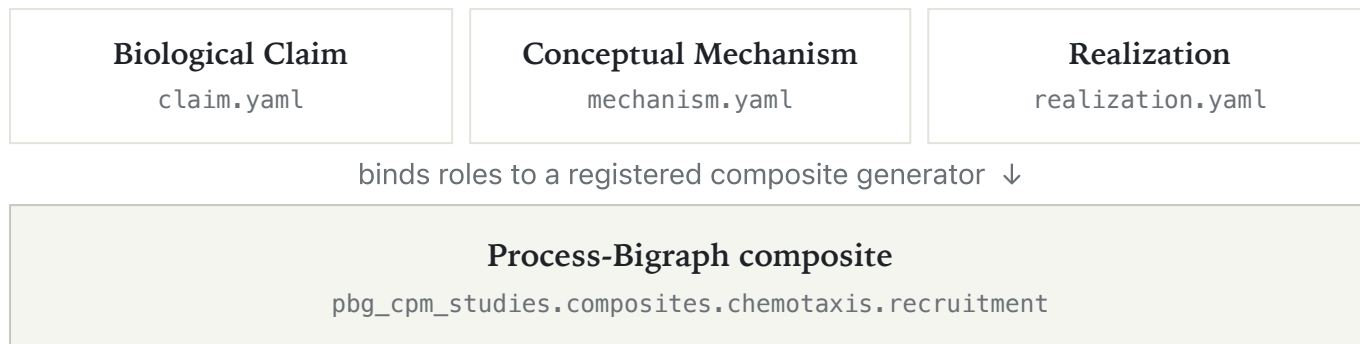


Figure 1. The three backend-neutral documents sit above the executable composite; only the realization names a backend.

5 THE GROUNDING

Reality map: every pillar, and where it already lives

Most of the semantic layer is naming and single-sourcing what the workbench already derives.

- **SHIPS** already running
- **THIN ADD** formalize existing signals
- **NET-NEW** build on existing data
- **RESEARCH** out of RFC scope

PILLAR	STATUS	REAL HOME TODAY	WHAT IS MISSING
Claim / Mechanism / Realization documents	● THIN ADD	viva-cpm/.../claim-bundle/*.yaml	Prose YAML today; needs resolvable <code>\$schema</code> URIs and a migration harness (<code>study.yaml</code> already has one).
Traceability graph (claim ↔ study ↔ observable ↔ source)	● SHIPS	viva_superpowers/linkage_index.py ; needs_attention.py	Nothing structural. Add claim nodes to the existing index.
Behavior contracts (given / measure / expect)	● SHIPS	vivarium_workbench/lib/expected_behavior.py	Measures hard-wired to one state shape. Missing: measurement adapters, equivalence levels.
Scientific closure predicates	● THIN ADD	viva_superpowers/study_verdict.py	All signals exist; the four named predicates over them do not (§9).
Epistemic envelope + provenance	● SHIPS	finding_observations.py ; band_provenance.py	Value + units + range + divergence + cites already populated. Missing: schema URIs.
LLM-origin metadata + acceptance lifecycle	● SHIPS	investigation-contracts (RFC-0002)	Reuse its <code>Provenance</code> , <code>ACTOR_KIND</code> , and proposed→accepted lifecycle.

PILLAR	STATUS	REAL HOME TODAY	WHAT IS MISSING
Declared vs validated range; units	● SHIPS	<code>bigraph-schema</code> : <code>Range</code> , <code>units</code> , <code>Enum</code>	Expose range <i>tiers</i> on model settings.
Semantic sidecar on component ports	● NET-NEW	<code>Edge.contract</code> / <code>Edge.description</code>	No per-node <code>_meta</code> slot exists; use a path-keyed sidecar.
Claim → Process-Bigraph compiler	● RESEARCH	Absent — composites are hand-authored	The role→backend binding is a lossy human decision. Deferred (§11).

Takeaway. The layer is roughly **70% naming and single-sourcing** what already ships, **20% thin additions**, and **10% deferred research**. That is what makes it realistic.

6 DELIVERY

Roadmap

Four phases, each with a falsifiable gate demonstrated on the chemotaxis slice.

PHASE 1 Formalize & single-source	Versioned claim schemas; derive the prose Turn <code>biological-claim/0.1</code> into schema-validated documents with URIs and migrations. Make <code>claim.yaml</code> canonical; generate the study and investigation prose from it. GATE <i>the three studies' claim prose is generated from the bundle with zero drift, and the <code>intervention.realization</code> contradiction is flagged.</i>
PHASE 2 Closure & ranges	Name the predicates; tier the ranges Compute the four closure predicates from existing signals as a derived verdict. Add range tiers on <code>model_settings</code> backed by <code>bigraph-schema</code> <code>Range</code> and units. GATE <i>a code-computed closure verdict shows <code>claim_complete: false</code> for chemotactic recruitment, correctly, with its unresolved hole.</i>

PHASE 3 Provenance & linting	Acceptance lifecycle; realization linter Route claim and realization documents through <code>investigation-contracts</code> provenance and acceptance. Ship a linter: bindings must resolve to real composite-generator parameters. GATE <i>an agent-proposed claim is marked <code>actor: agentic</code> and cannot satisfy a required commitment until <code>accepted</code>.</i>
PHASE 4 Research direction	Template-expander → toward a compiler Deterministically emit study specs from a <i>typed</i> parameter-sweep realization (the <code>generated-studies</code> the bundle anticipates). Explore equivalence ladders and measurement adapters for cross-backend contracts. GATE <i>the three chemotaxis studies are emitted from the realization rather than hand-authored — semantic-gap annotations preserved.</i>

PART III

Worked case study — chemotactic recruitment

The real files, end to end, then the same slice once modeling is semantic-driven.

7 THE SLICE, IN FULL

From a biological question to a checked result

The biological question. If a cluster of cells secretes a diffusible cue into their surroundings, will other motile cells that can sense the cue move toward the source and accumulate there? And crucially — is that accumulation actually *caused* by the cue-and-response, or could it arise by chance? This is chemotactic recruitment, one of the most basic pattern-forming behaviors in multicellular biology. Below is exactly how it is represented today in `viva-cpm`, with the real files.

1 State the claim — what is asserted

claim.yaml

The claim is a backend-neutral assertion. It names the subject, relation, and object; the context it holds in; the intervention that should switch it off; the observable that measures it; and the alternative explanations it must beat, with the results that would falsify it.

```
schema: https://vivarium-collective.org/biological-claim/0.1
id: claim:chemotactic-recruitment
kind: causal-contribution
assertion:
  subject: secreted-chemotactic-cue
  relation: recruits
  object: responder-cells
intervention:
  protocol: block-chemotactic-response
  realization: unresolved # drift: realization.yaml already binds this
observable:
  quantity: recruitment-index
  measurement-model: fraction-of-responder-COMs-within-radius-of-source
prediction:
  relation: decreases
  output: recruitment-index -> 0 under the intervention
alternatives:
  - {id: random-diffusion-accumulation, status: excluded} # by the inhibited arm
  - {id: metric-proximity-artifact, status: excluded} # by the adversarial arm
epistemic:
  status: empirically-demonstrated
  effect-size: {cohens_d: 6.38, n_seeds: 5}
```

assertion	The scientific content in three tokens: a secreted cue <i>recruits</i> responder cells. Everything else scopes or tests it.
intervention	How to switch the effect off — the heart of a causal claim. (Note <code>realization: unresolved</code> here already contradicts <code>realization.yaml</code> , which binds it — the drift the layer catches.)
alternatives	The rival explanations that must be excluded. This is what makes it a scientific claim rather than a demonstration.

2 Describe the mechanism — what can happen

mechanism.yaml

The mechanism is the conceptual pattern that *could* realize the claim, expressed as roles and capabilities. It mentions no backend — no pixels, no lattice, no forces — only what each participant must be able to do.

```
id: mechanism:chemotactic-response
kind: ProcessSchema
roles:
  responder:
    requires: [spatially-located, motile, expresses-chemotactic-response]
  cue:
    requires: [diffusible-scalar-field]
environment:
  requires: [gradient-query, spatial-frame]
enabled_when:
  - responder expresses a compatible response capability
  - a resolvable cue gradient is present
  - the response is not inhibited
contributes:
  channel: directional-motility-bias
  operation: up-gradient-bias
  combination: additive # combines with persistence + random motility
controls: [inhibit, scale, reverse, replace-law]
```

roles · requires	A typed contract: any realization must supply participants with these capabilities. This is what a candidate composite gets <i>checked against</i> — the seam that lets meaning constrain implementation.
contributes	How the mechanism acts on the world: an up-gradient bias to motility, combined additively with other drives. Backend-neutral, but precise.
controls	The knobs a study can turn — <code>inhibit</code> is the one the intervention arm uses. Named here, bound in the realization.

3 Realize it — and record the honest gap

realization.yaml

The realization makes the mechanism executable in one backend (`viva-cpm`), binding each role to a concrete implementation and — critically — recording what the backend *cannot* express instead of hiding it.

```
claim: claim:chemotactic-recruitment
mechanism: mechanism:chemotactic-response
backend: viva-cpm
fidelity: real-backend          # bridges the actual Rust CPM engine
bindings:
  cue:
    implementation: "fields[].secretion - source cells emit a diffusible field C"
    fidelity: phenomenological
  responder-response:
    implementation: "fields[].chemotaxis lambda - a bias on the copy-attempt energy"
    fidelity: phenomenological
  recruitment-observable:
    implementation: "recruitment_index from cell centres of mass"
    fidelity: equivalent
  intervention-binding:
    block-chemotactic-response: "set the responder chemotaxis lambda to 0"
  unresolved:
    - id: hole:lambda-calibration
      consequence-if-unresolved: "prediction is qualitative/relative, not an absolute rate"
  semantic-gap:
    - "CPM chemotaxis lambda is a PHENOMENOLOGICAL response strength, NOT a
      receptor-level model. A receptor-kinetics realization would refine this."
  equivalence: {level: phenomenological-equivalent}
  alternative-realizations:
    - {backend: physicell, status: not-implemented}
    - {backend: receptor-kinetics-ode, status: not-implemented}
```

fidelity (per binding)	Each role records its own honesty label — the cue and response are <i>phenomenological</i> , the observable is <i>equivalent</i> . Fidelity is per-binding, not one blanket claim.
semantic-gap · equivalence	The load-bearing honesty. The λ is a stand-in for receptor kinetics; the realization is <i>phenomenological-equivalent</i> , not mechanistically equivalent. A machine can now read that, not just a careful reviewer.
alternative-realizations	The same claim, other backends — listed, not yet built. This is what backend-independence buys, made explicit.

4 Design the experiment — prove causation, not correlation

three studies

One claim is tested by three studies that all run the *same* composite, `chemotaxis.recruitment`, at different parameters. Together they form a causal-necessity argument: the effect appears only when both cue and response are present, and vanishes when either is removed.

STUDY	ROLE IN THE ARGUMENT	CUE_RATE	CHEMO_LAMBDA	BEHAVIOR TEST
recruitment-baseline	positive — cue and response present	10.0	14.0	final gt 0.5
recruitment-inhibited	intervention — response blocked ($\lambda \rightarrow 0$)	10.0	0.0	max lt 0.1
recruitment-adversarial	negative control — no cue emitted	0.0	14.0	max lt 0.1

The *inhibited* arm excludes "cells just diffuse together anyway"; the *adversarial* arm excludes "the metric counts them as close by chance." Those are precisely the two `alternatives` the claim listed in Step 1.

5 Measure it — one backend-neutral observable

metrics.py

The observable `recruitment-index` is defined conceptually in the claim and implemented as a pure function over emitted run data — the fraction of responder cells whose centre of mass is within a radius of the source centroid. The behavior-contract evaluator calls this on recorded frames; a study passes when the measured value clears its declared band.

```
def recruitment_index_from_coms(coms, types, *, radius=15.0) -> float:
    """Fraction of responder cells within `radius` of the source centroid."""
    src, resp = _split(coms, types, SOURCE_TYPE, RESPONDER_TYPE)
    if not resp or not src:
        return 0.0
    c = _centroid(src)
    within = sum(1 for r in resp if _dist(r, c) <= radius)
    return within / len(resp)
```

Because the observable is a pure function of positions — not of CPM internals — the *same conceptual measurement* could be computed from a PhysiCell run or an ODE readout with a different adapter. That is the seam §11 builds on.

6 Read the result — findings, verdict, closure

runs · gate

The runs settle the argument cleanly. Baseline recruits strongly; both controls sit at zero — a very large effect (Cohen's $d \approx 6.4$ across five seeds).

STUDY	MEASURED	TEST	GATE
recruitment-baseline	0.767	final gt 0.5	● PASSED
recruitment-inhibited	0.000	max lt 0.1	● PASSED
recruitment-adversarial	0.000	max lt 0.1	● PASSED

Today the study records a bare `gate_status: passed`. The semantic layer replaces that scalar with a *derived* closure verdict that says what actually passed — and, honestly, what did not:

```
closure:
  claim: claim:chemotactic-recruitment
  well_formed: true           # schema-valid, references resolve
  claim_complete: false      # an unresolved hole remains (below)
  backend_realizable: true    # bindings resolve to a real generator
  execution_ground: true      # every readout is a real emit-plan leaf
  evaluated_by: code          # derived, never hand-set
```

7 Name what is still open

unresolved hole

The claim is demonstrated *qualitatively*, but the CPM `lambda` has no calibrated mapping to a real chemotactic sensitivity, so it cannot yet predict an absolute rate. The layer records this as a structured hole with its consequence — and this is what drives the next piece of modeling (§11).

```
unresolved:
- id: hole:lambda-calibration
  required_for: [quantitative-treatment-prediction]
  not_required_for: [recruitment-necessity]
  consequence: qualitative-only
```

What the case study demonstrates. Every part of the semantic layer already has a home in this one investigation — the claim, the mechanism, the honest realization, the causal experimental design, the backend-neutral observable, and the open hole. The layer's job is to make these *canonical, versioned, and machine-checkable* rather than hand-authored and drift-prone.

8 THE SAME SLICE, DRIVEN

How this works when modeling is semantic-driven

Today the composite is authored first and described after. Here the claim leads.

The steps above run "forwards" only because a person authored the composite and then wrote the claim around it. In a semantic-driven workflow the *claim is the source*, and the same slice unfolds differently. Three concrete moves, all reachable from the files above:

MOVE 1 · PORTABILITY

One claim, a second backend

The realization already names

`receptor-kinetics-ode` as an alternative.

Because `recruitment-index` is a backend-neutral observable with a pluggable adapter, the *same* behavior tests run against an ODE realization — and the two are compared at a declared equivalence level, not by pretending they are identical.

MOVE 2 · ACCUMULATION

The binding becomes reusable

"Responder-response → a motility-bias parameter, fidelity phenomenological, gap = no receptor kinetics" is authored once and lifted into a typed binding registry. The next chemotaxis-like claim reuses it — expansion becomes deterministic without re-deriving the modelling judgment.

MOVE 3 · DIRECTION

The hole pulls the next study

`hole:lambda-calibration` is *required* for a quantitative prediction. The layer proposes the next modelling action — calibrate λ against a known sensitivity, or add a receptor-kinetics realization — closing the loop instead of waiting for someone to notice.

The flip, concretely. A modeler writes (or an agent proposes) the *claim and mechanism* first. The layer asks the realization to satisfy the mechanism's `roles/requires`, checks the bindings resolve to a real generator, expands the causal experimental design (baseline · intervention · control) from the pattern, runs it, computes closure, and surfaces the remaining hole as the next action. The composite is now *answerable to the biology*, not the other way round.

PART IV

Reference & North Star

The mechanics, the long-term vision, the boundaries, and the corrections.

9 MECHANICS OF CLOSURE

Closure predicates over signals that already exist

The four predicates are a naming layer, not a new checker. Each maps to running code.

PREDICATE	QUESTION	COMPUTED FROM (EXISTS TODAY)
<code>well_formed</code>	Schema-valid, references resolve?	<code>bigraph-schema</code> <code>Core.check/validate</code> ; migration success.
<code>claim_complete</code>	All commitments specified, accepted, or excluded?	Realization <code>unresolved[]</code> + investigation <code>decisions_needed</code> + <code>open_epistemic_debts()</code> .
<code>backend_realizable</code>	A viable realization exists for a backend?	Bindings resolve to a registered <code>@composite_generator</code> and its parameters.
<code>execution_ground</code>	Every declared readout is a real emit-plan leaf?	The never-fabricate guard: <code>ReadoutRow.emit_status</code> / <code>build_study_observable_check</code> .

Keep the distinction 0.1 got right. Scientific holes are not runtime holes. `execution_ground` (can this run?) stays separate from `claim_complete` (is the science sufficiently specified?). A study can execute perfectly while its claim remains uncalibrated — exactly the chemotaxis slice's stated position.

10 WHERE THINGS LIVE

Grounded architecture

No new top-level package — a small schema set plus additions to packages that exist.

biological-claim/0.x	● NET-NEW A small versioned schema set (claim / mechanism / realization / closure), living beside the studies that use it in <code>claim-bundle/</code> .
viva_superpowers	● SHIPS Hosts the derivations (<code>linkage_index</code> , <code>needs_attention</code> , <code>study_verdict</code> , <code>finding_observations</code>). Adds claim nodes, the closure roll-up, the single-source derivation.
investigation-contracts	● SHIPS RFC-0002 typed nodes + <code>Provenance</code> + acceptance lifecycle. The home for LLM-origin metadata — reused, not duplicated.

bigraph-schema	• SHIPS Supplies <code>Range</code>, units, <code>Enum</code>, and validation. Backs range tiers and <code>well_formed</code>. No changes required.
vivarium-workbench	• THIN ADD Surfaces the closure verdict and claim view in the Investigations tab through existing endpoints.
process-bigraph	• UNTOUCHED The runtime. Annotations attach via a path-keyed sidecar or the no-op <code>Edge.contract</code> slot — never a per-node in-document field.

11 THE LONG GAME

North Star: from bookkeeping to driving

What it takes to move from describing models to letting biology drive them.

Everything in Parts I–III makes the semantic layer a faithful *description* of models that already exist. "Semantic-driven" means flipping the arrow: biological meaning becomes the **source**, and the computational realization becomes **answerable to it**. The mechanism's `roles/requires` stop being documentation and become a contract a realization must satisfy. Five capabilities make that flip real, ordered from achievable now to genuine frontier.

Observables as backend-neutral measurement contracts	
1	Promote each observable to one conceptual definition with a per-backend adapter (CPM extractor · PhysiCell extractor · ODE readout). The seam that lets one claim be tested across realizations — the operational meaning of backend-independent biology. NEAR-TERM
A typed realization-pattern registry — the binding library	
2	Do not build a magic compiler; accumulate reusable bindings ("role of kind K, in backend B, realized by W, fidelity F, gap G"), authored once with provenance. Expansion becomes deterministic for any claim reusing them — the generative engine. NEAR-TERM
Ontology-grounded referents	
3	Let claim terms resolve to real referents (Gene Ontology, Cell Ontology, ChEBI identifiers) instead of local strings. This turns the private claim graph into a node in the actual biology knowledge graph — comparable and composable across investigations. FRONTIER

Composable mechanisms

4

Compose mechanisms the way processes compose — chemotaxis $\oplus$ proliferation $\oplus$ adhesion $\rightarrow$ a tissue model — with the layer checking biological coherence (no conflicting roles, capabilities provided) before anything is realized.

FRONTIER

The hole-driven refinement loop

5

Closure feeds the next action: an unresolved hole with a consequence tells you what to model next. The argument graph pulls new studies into existence instead of you pushing them.

FRONTIER

The ratchet — how it scales past hand-authoring

These compose into a self-reinforcing loop, and this is where AI earns its place honestly — as a proposer, never an unchecked committer:



each accepted binding widens deterministic coverage — the loop tightens with use $\hookrightarrow$

Figure 2. The acceptance ratchet. Agents propose; a human or a test gates; accepted bindings accumulate; each one makes the next model cheaper to realize.

The honest limit. The binding is an *irreducible modelling judgment*. One cannot derive receptor kinetics from the word "recruits" — the grain of a model is the modeler's choice, and the artifacts are right to label their own semantic-gap. So the North Star is not "type a claim, get a validated model." It is: **biological meaning becomes the primary, ontology-grounded, composable, first-class artifact that constrains, selects, checks, and accumulates realizations — with every lossy step made explicit, reusable, and gated.** Rungs 1, 2, and 5 reach *semantic-driven*; rungs 3 and 4 reach *true biological*.

12 CONCEPT LINEAGE

The fuller model behind this RFC

This RFC scopes a subset. The source design notes define a richer conceptual model; its load-bearing taxonomies are retained here for reference.

The distinctions below come from the design notes this specification derives from. Each is deliberately *larger* than what RFC 0.2 implements; §5 grounds the implemented subset against shipping code. They are recorded so the layer grows toward a coherent whole rather than an ad-hoc set of fields.

Three transformations, not one "compiler"

The path from a claim to executable code is three distinct steps. Calling them all "compilation" would conceal the scientific judgment in the first two:

TRANSFORMATION	FROM → TO	CHARACTER
Interpretation	biological assertion → candidate mechanisms	Ambiguous; may need an LLM, expert judgment, and multiple retained alternatives.
Realization planning	mechanism → components, adapters, backend strategy	A constrained search; preserves alternatives where several implementations are plausible.
Deterministic lowering	accepted plan → Process-Bigraph / CC3D / PhysiCell / code	As deterministic and testable as an ordinary compiler.

Only the third step is a "compiler" in the ordinary sense; §11's template-expander is its first achievable slice. The first two are epistemic and stay agentic-and-gated.

Four contract faces

Today's typed port is one face of a fuller component contract. The meaning layer adds three more, and compatibility becomes a pipeline rather than a single check:

FACE	CARRIES	STATUS TODAY
Data	Type, shape, units, update algebra, default, serialization	● SHIPS — the current typed port
Semantic	Biological referent, quantity vs latent state, spatial frame, orientation, allowed ambiguity	● NET-NEW
Affordance & control	Queries, observations, accepted contributions, controls, legal interaction sequences, adapters	● NET-NEW
Assurance	Assumptions, range tiers, uncertainty, failure modes, conformance tests, evidence, advertised vs demonstrated	● PARTIAL — via findings / bands

Compatibility then runs structural → semantic → protocol → range → claim-relative adequacy → conformance, performed by a *contract broker* layered above the runtime's structural `admits` — not by overloading it.

The epistemic envelope

Rather than storing `migration-speed: 0.8`, a value carries its epistemic status and its range tiers, so an approximation cannot silently read as a measurement:

```

migration_speed:
  value: 0.8
  units: micrometer/minute
  status: inherited          # one of ~14 first-class statuses
  ranges:
    software_accepted: [0, 20]
    numerically_tested: [0, 8]
    biologically_plausible: [0.2, 3]
    validated: [0.5, 1.2]

```

Statuses are first-class — asserted, inherited, assumed, LLM-proposed, negotiated, calibrated, empirically-demonstrated, externally-validated, contradicted, unknown, contested, excluded, unsupported. A single `confidence` float is asked to carry too many distinct meanings.

Equivalence and fidelity are qualified, never assumed

When two backends realize the same claim they are compared at a declared level, and each realization records how faithfully it implements the mechanism. A compiler that emits code without a *semantic-gap report* is not adequate for scientific compilation.

EQUIVALENCE LEVELS

mechanistic · mathematical · numerical · statistical (for named observables) · phenomenological · not-known-to-be-equivalent.

IMPLEMENTATION FIDELITY

native realization · real-simulator bridge · equivalent reimplementation · numerical approximation · behavioral surrogate · phenomenological proxy · mock · unsupported.

Predicates, and a required hard slice

The notes carry a fifth closure predicate beyond §9's four — `evidence_sufficient(M, K, U)`, parameterized by claim K, backend B, and intended use U: a model validated for one claim can be inadequate for another. They also insist the compiler be proven on a *hard* slice, not only an easy one:

- **Easy slice (this document).** Chemotactic recruitment — a signal-behavior relation every backend can express.
- **Hard slice (required).** Apical constriction or planar polarity — forcing the system to identify a missing backend concept, propose alternatives, state a semantic gap, and *reject an invalid compilation*. A compiler tested only on constructs every backend already shares will look far more successful than it is.

13 BOUNDARIES

Explicit non-goals

- **A compiler that lowers an English biological claim into Process-Bigraph.** The role→backend-parameter step is a lossy modelling judgment the artifacts label as a semantic gap. Research, not RFC 0.2. The achievable near-term is the template-expander over an *already-typed* realization.

- A new type system parallel to `bigraph-schema`. Ranges, units, enums, and validation already exist there. Reuse them.
- A per-node `_meta` slot inside composite documents. Underscore keys are stripped as schema keys. Annotations live in a path-keyed sidecar or the no-op `Edge.contract` class slot.
- A standalone autonomous contract broker. 0.2 ships an inspector (`describe` first), not a negotiator.
- Redesigning `Process`, `Composite`, the scheduler, or site/fill semantics. A core runtime change is proposed only after a slice proves an upstream or sidecar solution cannot meet a concrete requirement.

14 APPENDIX

Corrections carried from 0.1

Claims in the prior draft that the codebase contradicts.

0.1 ASSUMED	REALITY
Claim / Mechanism / Realization is a greenfield design	Already prototyped on disk as <code>biological-claim/0.1</code> for chemotactic recruitment.
A compiler will lower claims into Process-Bigraph	Absent; the reference slice's own README calls it future work. Binding is a human decision.
Semantics attach via a no-op <code>_meta</code> node slot	No such slot; underscore keys are stripped. Use a sidecar or <code>Edge.contract</code> .
A concepts document defines the on-disk shapes	Canonical shapes live in <code>scaffold_yaml.py</code> and <code>study_spec.py</code> .
Packages <code>pbq-marketplace</code> / <code>pbq-superpowers</code> / <code>pbq-template</code>	Real names: <code>viva-marketplace</code> / <code>viva-superpowers</code> / <code>process-template</code> .
Behavior contracts and traceability are to-be-built	Both ship: <code>expected_behavior.py</code> and the linkage index / needs-attention scan.

Draft RFC 0.2 · Reference slice: chemotactic recruitment (viva-cpm)

Grounded against the working tree of `viva-cpm`, `vivarium-workbench`, `viva-superpowers`, `investigation-contracts`, `process-bigraph`, and `bigraph-schema`. Framework framing per Agmon & Spangler, *Process Bigraphs and the Architecture of Compositional Systems Biology* (arXiv:2512.23754). Conceptual model and the mappings in §1–§2 and §12 distilled from the design notes *LLMs in Biological Modeling*. Status marks reflect code that exists today, not aspiration.